

CS F425 Deep Learning Course Project

Building an AI Agent for Data Analysis
Description v1.0

Note:

- Please read the entire description carefully.
 - Further details for this project will be released in phases up to the deadline.
-

Objective

In this project, you will build an AI agent capable of solving structured data analysis tasks by interacting with a set of predefined tools. The agent should be able to interpret a natural language query, decide on a sequence of actions, use the available tools, and return the final answer.

Task Description

You are given:

- Sales table data (in CSV format)
- A set of tools (APIs) that operate on the dataset
- A tool executor program where the tools can be executed
- A dataset of input queries and a possible sequence of actions

Your goal is to train an AI agent that can:

1. Understand a user query expressed in natural language
2. Decide which tools to use
3. Determine the correct sequence of tool usage
4. Execute the tools to compute the final result
5. Return the final answer (and optionally intermediate steps)

Available Tools

The agent has access to the following tools:

- Filtering data based on conditions
- Grouping data by columns
- Aggregating values (sum, mean, count)
- Sorting results
- Selecting top-k entries
- Plotting results

Detailed documentation of these tools is provided in the starter code.

Agent Input–Output Specification (Strict)

The behavior of your agent must follow a clearly defined input–output format.

Input: A natural language query describing a data analysis task.

Output: Your agent must return a JSON object with the following structure:

```
{
  "actions": [...],
  "answer": ...
}
```

- **actions:** A list of steps taken by the agent.
- **answer:** The final result after executing the steps.

Important:

- Each element in **actions** must be a valid tool invocation using the provided APIs.
- Each step must follow the exact syntax:

```
tool_name(arg1=value1, arg2=value2, ...)
```

- The output must be valid JSON.
- No extra text, explanation, or formatting is allowed outside the JSON.

Examples

Example 1: Input:

What is the total revenue in 2022?

Output:

```
{
  "actions": [
    "filter_data(column='year', value=2022)",
    "aggregate_sum(column='revenue')"
  ],
  "answer": 52345678.12
}
```

Example 2: Input:

What is the average profit by region?

Output:

```
{
  "actions": [
    "group_by(column='region')",
    "aggregate_mean(column='profit')"
  ],
  "answer": {
    "North": 4200.5,
    "South": 3800.2,
    "West": 5100.8,
    "East": 2900.1
  }
}
```

Example 3: Input:

Top 3 cities by revenue in 2023

Output:

```
{
  "actions": [
    "filter_data(column='year', value=2023)",
    "group_by(column='city')",
    "aggregate_sum(column='revenue')",
    "sort_by(column='revenue', order='desc')",
    "top_k(k=3)"
  ],
  "answer": [
    {"city": "Mumbai", "revenue": 1234567},
    {"city": "Delhi", "revenue": 1132456},
    {"city": "Bangalore", "revenue": 1023456}
  ]
}
```

Example 4: Input:

Total profit for Electronics category in Mumbai

Output:

```
{
  "actions": [
    "filter_data(column='category', value='Electronics')",
    "filter_data(column='city', value='Mumbai')",
    "aggregate_sum(column='profit')"
  ],
  "answer": 8456231.55
}
```

Allowed Modifications

You are expected to build a working solution using the provided components. You may:

- Fine-tune a pre-trained language model using the provided training dataset
- Modify how the input query is formatted before being passed to the model (prompt design)

Not Allowed:

- Modifying the provided tools or their implementation

Generalization Requirement

Your agent will be evaluated on:

- A **held-out set of queries** not included in the training dataset
- A **different sales data CSV** have same format but different values.

Implication:

- Solutions that rely on memorizing answers from the training data will not perform well.
- Your agent must learn to correctly use the provided tools to compute answers.

Bonus tasks

TBA Note: completing these is necessary to get the total marks for the project.

Evaluation Criteria

TBA

Submission instructions

TBA

Note

- All training must be performed using free-tier GPU resources (Google Colab or Kaggle).
- You may use open-source pre-trained language models.
- You will have to submit your trained/finetuned model and inference scripts. The submitted model should train and inference on the above free-tier GPUs within 3-6 hrs.
- A good portion of the total marks is allotted to "bonus tasks" in addition to the above problem description. These will be released closer to the submission deadline.